

予約リマインダー・フォローメッセージ 送信動作 監査レポート

作成日: 2026-09-11 / 対象: 店舗情報編集の「予約リマインダー」「フォローメッセージ」設定と、実際の LINE 送信の仕組み

1. 結論（先に要点）

項目	判定	一言で
コード上のロジック（何日前・送信時刻・基準日の計算）	問題なし	設定どおりの日・時刻に送る計算になっている
本番の pg_cron が「毎時」になっているか	要確認（最有力の原因候補）	毎時に更新するマイグレーションは「Dashboard から手動実行」前提。未実施なら 19:00 以外の送信時刻は永久に送られない
cron から Edge Function を呼ぶときの鍵・タイムアウト	要確認	鍵が古い/新形式だと 401 で全件不送信。タイムアウト 5 秒で途中打ち切りの可能性
リマインダーの送信記録	仕組み上なし（改善推奨）	「送ったか送っていないか」を後から確認できない
希望日時（第一～第三希望）モードの予約	仕様上の注意	第一希望日を基準にリマインドが送られる
フォローメッセージ	問題なし	予定行が残るので追跡可能。ON にした後の予約から対象

「設定した時刻に送られていないかも」の原因として最も可能性が高いのは、Supabase 側の cron 登録（本番）が「毎日 10:00 UTC = 19:00 JST の 1 回」のまま、または鍵の不一致で Function が 401 を返している、のどちらかです。コードは正しいので、6 章の確認 SQL を本番の SQL Editor で実行して結果を教えてください。それで原因が確定します。

2. 確認したもの

対象	ファイル / 場所
設定 UI	src/app/tenant/[tenantSlug]/admin/[storeId]/page.tsx（店舗編集ダイアログ）
設定の保存	PUT /api/stores/{storeId}（src/app/api/stores/[storeId]/route.ts）
リマインダー送信	Edge Function supabase/functions/send-reminders/index.ts（staging v13 / 本番 v6、いずれも 2026-08-28 更新 = 最新コード）

対象	ファイル / 場所
フォロー送信	Edge Function <code>supabase/functions/send-follow-messages/index.ts</code> (staging / 本番とも v1、同一ハッシュ)
予定作成	<code>src/lib/follow-message-repository.ts</code> / <code>src/lib/follow-message-scheduler.ts</code>
cron 定義	<code>supabase/migrations/20250102000000_prod_cron_job.sql</code> (毎日 10:00 UTC) → <code>20260411100001_update_cron_hourly.sql</code> (毎時) → <code>20260908000000_add_follow_message.sql</code> (毎時 5 分)
予約データの入り方	POST <code>/api/reservations</code> 、静的フォーム <code>src/lib/static-generator-reservation.ts</code>

確認できなかったもの: 本番・staging の DB の実際の中身 (`cron.job` の登録内容、`net._http_response` の応答、`stores` の設定値)。CLI の保存トークンを使った読み取りが安全機構でブロックされたため、6 章の SQL をお願いする形にしています。

3. 送信の仕組み (おさらい)

3-1. 予約リマインダー

pg_cron (毎時 0 分に起動する想定)

→ Edge Function `send-reminders` を HTTP で呼ぶ (Authorization: Bearer <Service Role Key >)

1. 現在時刻 (JST) の「HH:00」を求める
2. `stores` から `reminder_enabled = true` かつ `reminder_time = その HH:00` の店舗だけ取得
3. 店舗ごとに「今日 + `reminder_days_before` 日」の日付を計算
4. その日付の予約 (キャンセル以外・`line_user_id` あり) を取得して 1 件ずつ push
5. 送信結果はログ (console) に出すだけ。DB には残さない

- 判定は「Function が実行された時刻の“時”」と `reminder_time` の**完全一致**です。cron が毎時動いていなければ、その時刻は一生来ません。
- 「何日前」は JST で正しく計算されています (`getStringJstAfterDays`)。月末・年末もライブラリの `setDate` が繰り上げるので問題ありません。

3-2. フォローメッセージ

予約成立 (POST /api/reservations)

→ follow_messages に 1 行 (基準日 + N 日 の HH:00 JST を scheduled_at に保存)

pg_cron (毎時 5 分)

→ Edge Function send-follow-messages

scheduled_at <= 今 の未送信行を取り、見送り判定 (再予約・リマインダー同日・OFF・キャンセル) → push → sent / skipped / failed を DB に記録

- 予定時刻を DB に持っているので、cron が 1 回止まっても次の回にまとめて送られます (リマインダーとの違い)。

4. 見つかった問題・リスク (重要度順)

4-1. 【高】本番の cron が「毎時」になっていない可能性

- 初期の cron は **毎日 10:00 UTC (19:00 JST) 1 回** で登録されています (20250102000000_prod_cron_job.sql)。
- 毎時に変えるマイグレーション (20260411100001_update_cron_hourly.sql) は、中に 'Bearer <SERVICE_ROLE_KEY>' という**プレースホルダー**が入っており、ファイルのコメントにも「実際の運用では Dashboard の SQL Editor から Service Role Key を直接指定して実行すること」とあります。つまり **supabase db push** で流しても本物の鍵にはならず、**手動で登録し直していない限り毎時にはなっていません**。
- この状態だと:
 - reminder_time を **19:00 以外**にした店舗 → **一度も送られない** (19:00 に起動したとき reminder_time = '19:00' の店舗しか一致しない)
 - reminder_days_before は 19:00 の起動時に計算されるので、19:00 の店舗だけは「何日前」どおりに送られる
- 「時刻を変えたら届かなくなった」「19:00 の店舗は届く」という症状なら、ほぼこれです。

確認方法: 6-1 の SQL。send_reservation_reminders の schedule が 0 * * * * なら OK、0 10 * * * なら未更新です。**直し方:** 6-1 の直後に載せた cron.schedule を本番の Service Role Key で実行 (同じジョブ名で上書きされます)。

4-2. 【高】cron が使っている鍵で Function が 401 になっている可能性

- 両 Function とも verify_jwt: true (デプロイ情報で確認)。cron の Authorization ヘッダーには **JWT 形式の service_role キー (eyJ... で始まる)** が必要です。

- 次の場合は毎回 401 で **全店舗・全件が送られません**:
 - プレースホルダー `<SERVICE_ROLE_KEY>` のまま登録されている
 - Supabase の新形式 API キー (`sb_secret_...`) を入れた (JWT ではないため検証に失敗)
 - キーをローテーションした
- Function 側のログには 401 は残らず (Function 本体に到達しない) 、**cron 側の応答テーブル `net._http_response` にだけ残ります**。

確認方法: 6-2 の SQL。 `status_code` が 401 / 403 なら鍵の問題、200 なら OK、行が無ければ cron が動いていません。

4-3. 【中】cron → Function の HTTP タイムアウト (既定 5 秒)

- `net.http_post` は `timeout_milliseconds` を省略すると **5000 ms** で打ち切ります。リマインダーは店舗数 × 予約数だけ LINE API を順番に呼ぶ (1 件 200~500 ms) ため、対象が 10 件を超えると 5 秒を超えます。
- 打ち切られても Edge Function 側は処理を続ける実装ですが、実行環境の都合で途中終了する可能性は否定できません。**同じ時刻の店舗が多い場合に「後ろの店舗だけ届かない」症状**が出たら、これを疑ってください。
- 対策: cron の `net.http_post(...)` に `timeout_milliseconds := 60000` を追加 (6-1 の登録 SQL に含めてあります) 。

4-4. 【中】リマインダーは「送った記録」を残さない

- 成功・失敗が `console.log` だけで、Edge Function のログは Dashboard で数日分しか見られません。「本当に送られたか」を後から店舗ごとに確認できず、今回のような調査が難しくなっています。
- フォローメッセージは `follow_messages` に `sent / skipped / failed` と理由・エラーを残す作りにしたので、同じ仕組み (`reminder_logs` テーブル、または `follow_messages` を種類付きで共用) をリマインダーにも入れることを推奨します。二重送信の防止にもなります。

4-5. 【中】希望日時 (第一〜第三希望) モードの予約への送り方

- 希望日時モードの予約は `reservation_date` に**第一希望日**が入ります (静的フォームが `selectedDate` を送信)。店舗が第二希望で確定しても DB の日付は変わらないため、**第一希望日を基準にリマインドが送られます**。
- また、ステータスが `pending` (未確定) でも送られます (`status != 'cancelled'` のみで絞っているため) 。
- 「確定した日時で送る」には、店舗管理画面の予約編集で日時を第二希望に直す運用か、`confirmed` のみ送る設定の追加が必要です。現状は仕様として把握しておいてください。

4-6. 【低】LINE チャネルアクセストークンが空文字の店舗

- 店舗作成時は `line_channel_access_token: ''`（空文字）で保存されます。リマインダーの店舗抽出は `IS NOT NULL` なので空文字の店舗も拾い、Function 内で `if (!token)` により黙ってスキップされます。動作上は害がありませんが、**トークン未設定の店舗は管理画面で ON になっていても送られませんが**。設定画面でトークン未設定の警告を出すと親切です。

4-7. 【低】その他の確認済み事項（問題なし）

観点	結果
設定の保存	店舗編集ダイアログは店舗オブジェクト全体を <code>PUT</code> し、 <code>reminder_enabled / reminder_time / reminder_days_before / reminder_template / follow_*</code> が保存される。 <code>follow_*</code> はサーバーで値検証あり
送信時刻の書式	UI は <code>07:00 ~ 21:00</code> の <code>HH:00</code> だけを選ばせる。Function 側も <code>HH:00</code> で比較。不一致は起きない
タイムゾーン	両 Function とも JST で日付・時刻を計算。 <code>reservation_date</code> は DATE 型（JST の日付）。DB の <code>timezone</code> 設定に依存しない
複数店舗で時刻が違う	店舗ごとに <code>reminder_time</code> を判定し、予約は店舗ごとの対象日で振り分ける。混線しない
何日前の範囲	UI は 1~7 日、Function は 1~30 を許容。不正値は 1（前日）に丸める
キャンセル済み	両方とも <code>cancelled</code> は除外
Web 予約フォーム	<code>line_user_id</code> が無いので LINE 送信の対象外（仕様。メール通知のみ）
フォローの二重送信	<code>attempt_count</code> の樂觀ロックで防止。リマインダーは <code>cron</code> が 1 時間に 1 回だけ動く前提で防止（4-4 参照）
フォローの対象範囲	店舗で ON にした 後に成立した予約 だけが対象。ON 以前の未来予約には送られない（仕様）
過去日の手動登録	フォローの予定が過去になる場合は作らない（遅れて「ありがとうございました」を送らない）

5. 「設定どおりに送られる」ための前提条件チェックリスト

本番で以下がすべて満たされている必要があります。

- ☐ `cron.job` に `send_reservation_reminders` が `0 * * * *` で登録されている（4-1）

- □ `cron.job` に `send_follow_messages` が `5 * * * *` で登録されている
- □ 両ジョブの `Authorization` が **JWT 形式の service_role キー**（`eyJ` 始まり）で、`net._http_response` の直近の `status_code` が 200（4-2）
- □ `stores` に `reminder_days_before` / `reminder_template` / `follow_*` の列がある（無いと Function の店舗取得が失敗して 0 件になる）
- □ 対象店舗の `reminder_enabled = true`、`reminder_time` が `HH:00`、`line_channel_access_token` が空でない（4-6）
- □ 対象予約に `line_user_id` が入っている（LIFF 経由の予約）

6. 本番で実行してほしい確認 SQL（読み取りのみ・安全）

Supabase Dashboard → プロジェクト **wakuwakusystemsharing-prod** → SQL Editor で、上から順に実行して結果を共有してください。

6-1. cron の登録内容（最重要）

```
select jobname, schedule, active,  
       regexp_replace(command, 'Bearer [^']*+', 'Bearer ***') as command  
from cron.job  
order by jobname;
```

期待値: `send_reservation_reminders = 0 * * * *`、`send_follow_messages = 5 * * * *`、両方 `active = true`。`send_reservation_reminders` が `0 10 * * *` なら、次を **本番の service_role キー（JWT 形式）** に置き換えて実行してください（同名なので上書きされます）。

```

select cron.schedule(
  'send_reservation_reminders',
  '0 * * * *',
  $$
    select net.http_post(
      url := 'https://tpuqjpdasxfwsvjcbum.supabase.co/functions/v1/send-reminders',
      headers := jsonb_build_object(
        'Content-Type', 'application/json',
        'Authorization', 'Bearer ここに本番の service_role キーを貼る'
      ),
      timeout_milliseconds := 60000
    );
  $$
);

```

6-2. cron が呼んだときの Function の応答（直近）

```

select id, status_code, left(content, 120) as content, created
  from net._http_response
 order by created desc
 limit 20;

```

期待値: status_code = 200、content が {"sent":N,"errors":0} や {"sent":0}。

- 401 / 403 → 鍵が違う（4-2）。6-1 の登録 SQL を正しい鍵で実行し直す
- 行が無い / 古い → cron が動いていない、または pg_net 拡張が無効
- timed out を含む → 4-3。timeout_milliseconds を付けて登録し直す

6-3. cron の実行履歴

```

select j.jobname, d.status, d.start_time, left(d.return_message, 80) as msg
  from cron.job_run_details d
 join cron.job j on j.jobid = d.jobid
 order by d.start_time desc
 limit 30;

```

期待値: 1 時間ごとに succeeded が並ぶ。1 日 1 回しか無ければ 4-1。

6-4. 店舗の設定値と列の存在

```
select id, name, reminder_enabled, reminder_time, reminder_days_before,  
       (line_channel_access_token is not null and line_channel_access_token <> '') as ha  
s_token,  
       follow_enabled, follow_base, follow_days_after, follow_time  
from stores  
order by name;
```

期待値: エラーなく一覧が出る（列が無いとここでエラー = マイグレーション未適用）。送りたい店舗は `reminder_enabled = true`、`has_token = true`。

6-5. 今日・明日の送信対象になる予約

```
select s.name as store, s.reminder_time, s.reminder_days_before,  
       r.reservation_date, r.reservation_time, r.status,  
       (r.line_user_id is not null) as has_line  
from reservations r  
join stores s on s.id = r.store_id  
where r.reservation_date between current_date and current_date + 7  
and r.status <> 'cancelled'  
order by r.reservation_date, s.name;
```

「届かなかった」と言われた予約が `has_line = true` で、その店舗の `reminder_days_before` 日前の `reminder_time` に cron が動いていたか（6-3）を突き合わせると原因が特定できます。

6-6. フォローメッセージの状況

```
select status, skip_reason, count(*)  
from follow_messages  
group by status, skip_reason  
order by status;
```

7. 推奨する対応（優先順）

1. 6-1～6-3 の結果で cron と鍵を正す（コード変更不要。SQL だけ）

2. cron の `net.http_post` に `timeout_milliseconds := 60000` を付ける (6-1 の SQL に含む)
 3. **リマインダーの送信ログを DB に残す** (`reminder_logs` テーブル + Function で記録。二重送信防止と調査のため。半日～1 日の作業)
 4. 店舗編集画面で「LINE チャネルアクセストークン未設定」のときリマインダー / フォローの横に警告を表示 (小)
 5. 希望日時モードの予約を `confirmed` のみリマインドする設定の追加を検討 (要相談)
 6. マイグレーションの cron テンプレートに `timeout_milliseconds` を追加し、「手動登録が必要」であることをリリース手順に明記 (ドキュメント)
-

8. 補足: よくある誤解

- 「**保存したのに反映されない**」 → 保存は即時に DB に入ります。反映されないのは「次に cron が起動して `reminder_time` と一致した時」に初めて送られるため、保存直後に送られるものではありません。
- 「**前日 19:00 に設定したのに当日に届いた**」 → 4-1 の状態で `reminder_days_before = 1` なら前日 19:00 に届きます。当日に届く場合は `reminder_days_before` が 0 扱い (Function は 1 に丸めるので起きない) ではなく、予約日そのものが「翌日」で登録されている可能性があります。6-5 で予約日を確認してください。
- 「**フォローが届かない**」 → ON にする前に入った予約には予定が作られません。ON 後に新しく予約して `follow_messages` に行ができるか (6-6) を見てください。